

Linear Regression, Gradient Descent & Logistic Regression

Worked examples from codebasics' Machine Learning Tutorial Python series (videos 2–4), plus a standard logistic regression example

Part of the 3-Month Machine Learning Mentorship Program

These three ideas form the backbone of classical machine learning: linear regression predicts a number, gradient descent is how it actually learns that prediction, and logistic regression adapts the same machinery to predict yes/no outcomes instead. Every number and chart in this guide was computed and verified directly from the code shown — nothing here is estimated.

1. Linear Regression

1.1 What Is Linear Regression?

$$y = m \cdot x + b$$

Linear regression fits a straight line through data that best predicts a numeric output from one or more inputs. **m** (the slope) is how much *y* changes per one-unit increase in *x*. **b** (the intercept) is the predicted value of *y* when *x* is zero.

1.2 Worked Example: Home Price from Area

The running example throughout this guide, taken directly from the source tutorial: predicting a home's price from its area in square feet.

Area (sqft)	Price (\$)
2600	550,000
3000	565,000
3200	610,000

3600	680,000
4000	725,000

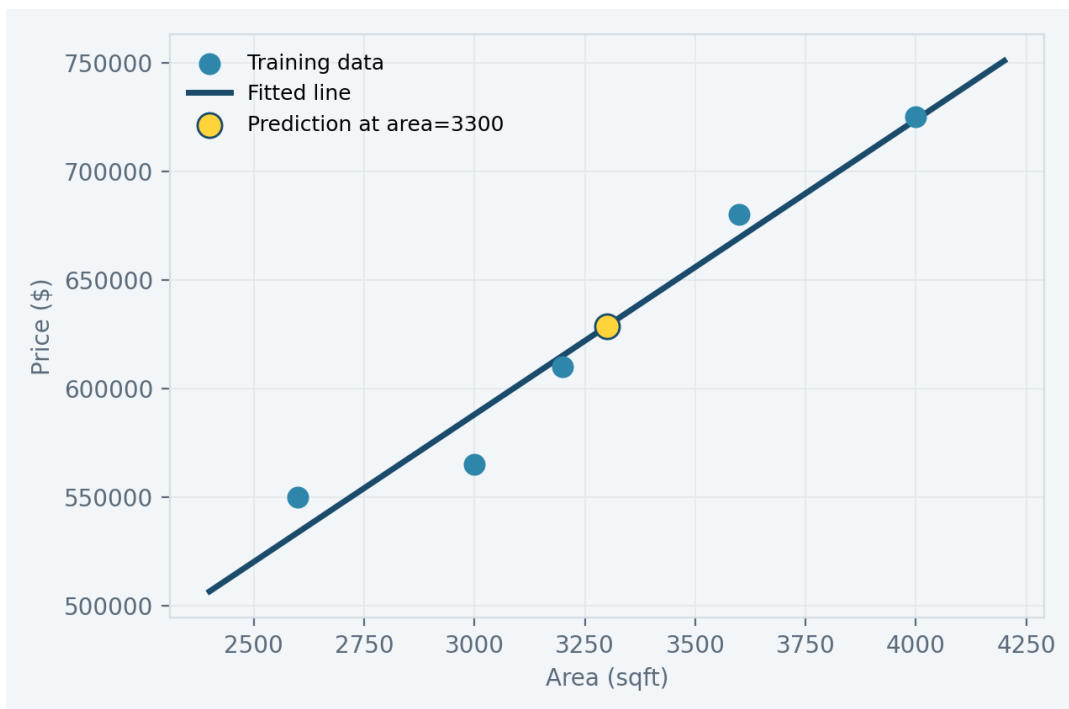
```
from sklearn.linear_model import LinearRegression

reg = LinearRegression()
reg.fit(df[['area']], df.price)

print(reg.coef_)      # 135.79
print(reg.intercept_) # 180616.44

reg.predict([[3300]]) # 628715.75
reg.predict([[5000]]) # 859554.79
```

The model learns: **price = 135.79 × area + 180616.44**



Training data (blue), the fitted line (dark), and a new prediction at area=3300 (yellow).

1.3 Adding More Variables

Real problems rarely have just one input. Extending the example to also use bedrooms and age means one more term (and one more coefficient) per feature — and one practical wrinkle: a missing bedrooms value to handle first.

$$\text{price} = m_1 \cdot \text{area} + m_2 \cdot \text{bedrooms} + m_3 \cdot \text{age} + b$$

```
import math

# one bedrooms value is missing (NaN) -- fill it with the median
median_bedrooms = math.floor(df.bedrooms.median()) # 3
df.bedrooms = df.bedrooms.fillna(median_bedrooms)

reg = LinearRegression()
reg.fit(df[['area', 'bedrooms', 'age']], df.price)

print(reg.coef_)      # [ 137.25 -26025. -6825. ]
print(reg.intercept_) # 383725.0

reg.predict([[3000, 3, 40]]) # 444400.0
```

2. Gradient Descent

2.1 The Cost Function

How does sklearn actually find m and b ? It minimizes a cost function called Mean Squared Error (MSE):

$$\text{MSE} = (1/n) \sum (y_i - (m \cdot x_i + b))^2$$

- MSE measures how far the line's predictions are from the actual values, on average.
- Squaring the errors penalizes big misses more than small ones, and keeps everything positive.
- Every possible (m, b) pair gives a different MSE — the “best” line has the lowest MSE.
- Gradient descent searches for that lowest point efficiently, without trying every possible line.

2.2 The Algorithm

Gradient descent follows four repeated steps:

- **1. Start** — begin with a random guess for m and b (often both 0)
- **2. Compute error** — measure the current MSE cost with these m, b values
- **3. Compute gradient** — find the partial derivatives of cost w.r.t. m and w.r.t. b
- **4. Update & repeat** — nudge m and b opposite the gradient; repeat until converged

$$m = m - \alpha \cdot (\partial \text{MSE} / \partial m) \quad b = b - \alpha \cdot (\partial \text{MSE} / \partial b)$$

α (alpha) is the learning rate — how big each step is.

2.3 Gradient Descent, From Scratch

A small dataset where the true pattern is $y = 2x + 1$ — watch gradient descent find it with no help:

```
def gradient_descent(x, y, learning_rate=0.08, iterations=1000):
    m_curr = b_curr = 0
    n = len(x)
    for i in range(iterations):
        y_pred = m_curr * x + b_curr
        cost = (1/n) * sum((y - y_pred) ** 2)
        md = -(2/n) * sum(x * (y - y_pred))    # d(cost)/d(m)
        bd = -(2/n) * sum(y - y_pred)         # d(cost)/d(b)
        m_curr = m_curr - learning_rate * md
        b_curr = b_curr - learning_rate * bd
    return m_curr, b_curr

x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])
m, b = gradient_descent(x, y)
print(m, b)    # 2.0000, 1.0000 -- found the true pattern!
```

2.4 Watching It Converge

Iteration	m	b	Cost
0	0.00	0.00	57.00
200	2.0005	0.9983	~0.00
400	2.0000	1.0000	~0.00
1000 (final)	2.0000	1.0000	~0.00

The key insight: m and b start at zero and get nudged, step by step, until the cost stops improving. This is exactly what sklearn's LinearRegression does internally — you just never see the loop.

3. Logistic Regression

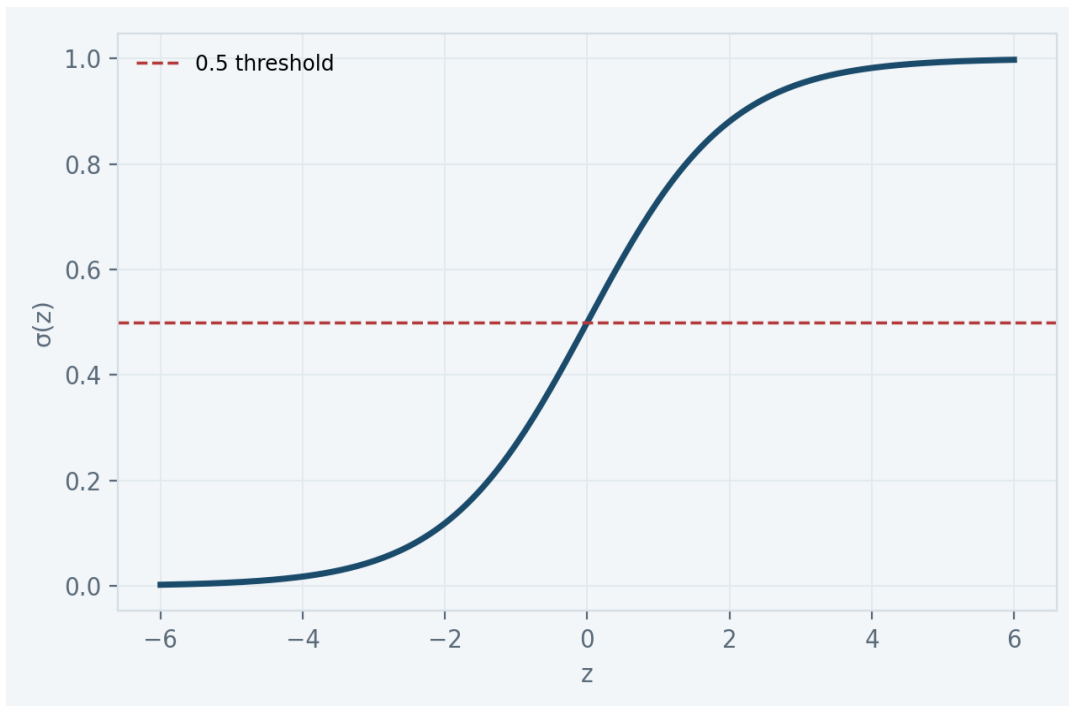
3.1 From Regression to Classification

What if we want to predict “yes/no” instead of a number? Linear regression's output isn't bounded — it can predict -50 or 300, which makes no sense as a probability. We need a function that squashes any input into a value between 0 and 1. Logistic regression uses the **sigmoid function** to do exactly that.

3.2 The Sigmoid Function

$$\sigma(z) = 1 / (1 + e^{-z})$$

z here is the familiar linear part: $z = m \cdot x + b$. As z grows large and positive, $\sigma(z)$ approaches 1; as z grows large and negative, $\sigma(z)$ approaches 0; at $z=0$, $\sigma(z)$ is exactly 0.5.



The sigmoid function: any input z is squashed into a probability between 0 and 1.

3.3 Worked Example: Pass/Fail from Study Hours

A standard example: predicting whether a student passes an exam, based on hours studied.

```

from sklearn.linear_model import LogisticRegression

# hours studied -> passed (0 = no, 1 = yes)
hours = [[0.5], [0.75], [1.0], [1.25], [1.5], [1.75], [2.0], [2.25], [2.5],
         [2.75], [3.0], [3.25], [3.5], [4.0], [4.25], [4.5], [4.75], [5.0], [5.5]]
passed = [0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1]

model = LogisticRegression()
model.fit(hours, passed)

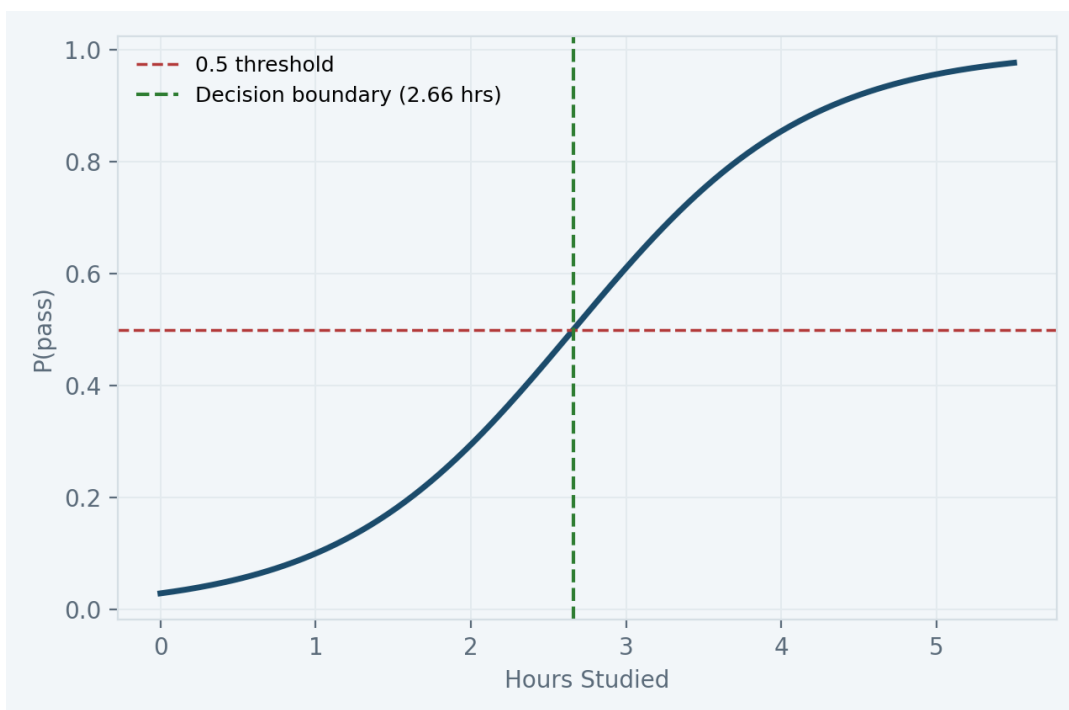
model.predict_proba([[1.5]]) # P(pass) = 0.177
model.predict_proba([[2.75]]) # P(pass) = 0.530
model.predict_proba([[4.0]]) # P(pass) = 0.855

model.predict([[1.5]]) # 0 -- predicted fail
model.predict([[4.0]]) # 1 -- predicted pass

```

3.4 The Decision Boundary

The model outputs a probability. Applying a threshold (usually 0.5) turns it into a yes/no decision — the point where the curve crosses that threshold is the decision boundary.



The fitted sigmoid curve for this example. The decision boundary sits at 2.66 hours studied.

4. Linear vs. Logistic Regression

Aspect	Linear Regression	Logistic Regression
Predicts	A continuous number	A probability (0 to 1), then a class
Output range	Unbounded ($-\infty$ to $+\infty$)	Bounded (0 to 1) via sigmoid
Example here	Home price from area	Pass/fail from study hours
Trained by	Gradient descent minimizing MSE	Gradient descent minimizing log loss

5. Summary & Key Takeaways

- Linear regression fits a straight line (or hyperplane) to predict a number from one or more features.
- Every extra feature just adds another term and another coefficient to learn.
- Gradient descent is how models actually learn: guess, measure error, adjust, repeat.
- Logistic regression adapts the same idea for yes/no predictions, using the sigmoid function.
- The decision boundary is simply the point where predicted probability crosses 0.5.

Linear regression, gradient descent, and logistic regression are the building blocks nearly every other ML algorithm extends in some way.

Further Resources

- **codebasics — Machine Learning Tutorial Python:** videos 2–4 cover linear regression and gradient descent in full, with the exact examples used here.
- **Companion slide deck:** LinearReg_GradientDescent_LogisticReg.pptx — the same material, presented visually.